Мастер-класс: инференс ML-моделей

От ноутбука до сервиса

Зачем говорить про инференс

После обучения модель обычно должна работать в окружении, где важны не только метрики качества, но и **стабильность**, **задержка ответа**, **пропускная способность** и **стоимость** вычислений.

Инференс — это этап, на котором модель превращается в часть системы, которая обслуживает запросы пользователей или других сервисов.

План занятия

1. Обучение небольшой модели и **экспорт в ONNX** (`flow/section_2`).
2. Сервинг **одного и того же ONNX** через **MLServer** и **Triton** (`flow/section_3`).
3. Отдельный блок про **LLM** и **vLLM** (`flow/section_4`).

Классификация изображений и генерация текста похожи на уровне идеи («модель принимает вход и возвращает выход»), но сильно различаются по устройству сервера и по работе с памятью.

Тайминг (ориентир)

По `Plan.md`:

- введение — около **7 минут**;
- модель и ONNX — около **13 минут**;
- серверы инференса — около **28 минут**;
- vLLM — около **8 минут**;
- итоги и вопросы — около **4 минут**.

Инференс и обучение

Обучение — веса модели обновляются по данным.
Инференс — веса фиксированы, модель многократно применяется к новым входам.

Для классификатора типичен один проход **forward**.
Для LLM — последовательная **генерация токенов**, накопление **KV-кэша** и более жёсткие требования к памяти при параллельных запросах.

Почему появляется инференс-сервер

В ноутбуке достаточно вызвать модель напрямую. В сервисе обычно нужны:

- приём и обслуживание **множества запросов**;
- **батчинг** и политики очередей;
- **версионирование** модели;
- проверки **готовности** и **живости**;
- **метрики** и наблюдаемость.

Инференс-сервер отвечает за эту «обвязку» вокруг вычислительного ядра модели.

Карта материалов

```
flow/section_2  -> ResNet-18 на CIFAR-10, экспорт в ONNX
flow/section_3  -> MLServer и Triton для одного ONNX
flow/section_4  -> vLLM и сравнение с HuggingFace
```

Блок 1: PyTorch → ONNX

В `flow/section_2/resnet18_cifar10.ipynb` используется **ResNet-18** на **CIFAR-10**:

- фиксированный вход изображения **3×32×32**;
- выход — **logits** для **10** классов;
- после обучения модель экспортируется в **ONNX**.

Смысл блока: ONNX становится **переносимым артефактом** с явным контрактом входов и выходов, который дальше можно запускать в разных рантаймах.

Зачем ONNX в этом мастер-классе

ONNX здесь — не отдельная «технология ради технологии», а **связующий формат** между ноутбуком и сервером.

В `section_2` ONNX получают экспортом.
В `section_3` тот же файл используют **MLServer** и **Triton**.

Контракт ONNX включает имена тензоров, типы данных и размерности — это то, что серверы используют как основу для API и конфигурации.

Блок 2: один ONNX, два сервера

В `flow/section_3` одна и та же модель запускается двумя способами:

- **MLServer**;
- **Triton Inference Server**.

Такой приём позволяет сравнивать не разные модели, а **разные подходы к сервингу** одной и той же модели.

MLServer: сервис вокруг модели

В `flow/section_3/mlserver/`:

- `run_mlserver_docker.sh` — запуск в Docker;
- `inference/model.py` — Python-обёртка над ONNX;
- `model-settings.json` — настройки загрузки модели.

MLServer хорошо иллюстрирует схему «модель + код + HTTP API», близкую к микросервисному развёртыванию.

MLServer: API и проверки

В `flow/section_3/client.ipynb` MLServer доступен по адресу `http://localhost:8080`.

Типовой набор операций:

- `GET /v2/health/live`
- `GET /v2/health/ready`
- `GET /v2/models/{name}`
- `POST /v2/models/{name}/infer`

В `model.py` используется **ONNX Runtime** с **CPUExecutionProvider** — это сознательный выбор для переносимости демонстрации.

Triton: сервер инференса и конфигурация модели

В `flow/section_3/triton/` используется **Triton Inference Server**.

Ключевые возможности, которые удобно подчеркнуть на примере:

- репозиторий моделей и **версии**;
- выбор **backend** (в демо — `onnxruntime`);
- управление **батчингом** и ресурсами через конфигурацию.

`config.pbtxt`: что там важно

В `flow/section_3/triton/model_repository/resnet18_cifar10/config.pbtxt` задаются:

- имя модели и backend;
- вход `image` и выход `logits`;
- `max_batch_size`;
- `dynamic_batching`;
- `instance_group`.

Этот файл — хороший пример того, что поведение сервера под нагрузкой задаётся **явно**, а не только кодом модели.

Dynamic batching

Dynamic batching позволяет серверу кратковременно накапливать запросы и выполнять их **батчем**, чтобы повысить утилизацию вычислителя.

Параметры вроде `preferred_batch_size` и ограничения по задержке очереди задают компромисс между **throughput** и **latency**: чуть большая задержка отдельного запроса может дать более эффективную обработку потока запросов.

MLServer и Triton: как сравнивать корректно

MLServer — сильная сторона в скорости упаковки модели в сервис и в гибкости Python-обвязки.

Triton — сильная сторона в «инференс-инфраструктуре»: версии, батчинг, разные backend-ы, типичный фокус на GPU-окружении NVIDIA.

В материалах мастер-класса важно подчеркнуть: сравнивается **инфраструктура**, а не «какая модель лучше».

`client.ipynb`: измерения

В `flow/section_3/client.ipynb`:

- используется один и тот же источник данных CIFAR-10;
- один и тот же вход прогоняется через MLServer и Triton;
- выполняются замеры для батчей `1, 2, 4, 8, 16, 32`.

На графиках удобно обсуждать **latency** и **throughput** и связь батча с эффективностью обработки.

Порты в демонстрации

В репозитории важно не перепутать порты:

- MLServer — **8080**;
- Triton — **8000** (и соседние служебные порты);
- vLLM по умолчанию — **8000**.

Одновременный запуск нескольких сервисов на одном порту приведёт к ошибке занятости порта — это нормальная ситуация для локальной демонстрации.

Блок 3: почему LLM — отдельная тема

В `flow/section_4` рассматривается генеративная модель.

Отличия от классификатора:

- переменная длина входа;
- пошаговая генерация;
- рост **KV-кэша**;
- более жёсткие требования к памяти при параллельных сессиях.

vLLM: что это даёт

vLLM ориентирован на сервинг декодирующих LLM:

- более эффективная работа с памятью (в т.ч. идеи уровня **PagedAttention**);
- **continuous batching** для потока запросов;
- **OpenAI-compatible HTTP API** (`/v1/chat/completions` и др.).

Это другой класс оптимизаций, чем сервинг фиксированного forward для картинки.

Материалы `flow/section_4`

- `run_vllm_docker.sh` — запуск vLLM в Docker на **CPU** (удобно для macOS / Docker Desktop);
- `run_vllm_docker_gpu.sh` — запуск на **GPU** в Linux с NVIDIA Container Toolkit;
- `curl_example.sh` — минимальный запрос к API;
- `vllm_demo.ipynb` — сравнение с наивным HuggingFace inference.

CPU и GPU для vLLM

CUDA-сборка vLLM ожидает доступ к GPU. В Docker на macOS GPU обычно недоступен в том виде, как на Linux-сервере, поэтому для демонстрации используется отдельный **CPU-образ**.

Для GPU-сценария используется отдельный скрипт с `--gpus all` и соответствующими настройками окружения.

Как интерпретировать сравнение в `vllm_demo.ipynb`

Сравнение локального HuggingFace inference и vLLM в контейнере может отличаться по:

- устройству вычислений (**CPU / MPS / CUDA**);
- накладным расходам Docker и сетевого слоя;
- режиму прогрева и параметрам генерации.

Корректный вывод: важно не только «кто быстрее в одном прогоне», а **какие факторы** влияют на результат и зачем вообще нужен отдельный LLM-сервер.

Итоги

1. **ONNX** связывает исследовательский код и серверный запуск.
2. **MLServer** и **Triton** показывают разные стили организации инференса для одной модели.
3. **LLM-сервинг** добавляет отдельный слой задач, связанных с памятью и планированием декодирования.

Куда смотреть дальше

- **TensorRT** — ускорение на стеке NVIDIA.
- **CUDA** — основа GPU-вычислений.
- **SGLang**, **Dynamo** — смежные направления вокруг LLM-сервинга.

Ссылки:

- Triton Inference Server
- MLServer
- vLLM

Типовые вопросы

Нужен ли только Triton?

Нет: выбор зависит от контекста развёртывания, требований к экосистеме и тому, насколько важна тонкая настройка инференс-сервера против скорости упаковки сервиса.

Заменяет ли vLLM Triton?

В рамках этого мастер-класса это разные уровни: классический сервинг модели с фиксированным входом и сервинг декодирующей LLM.

Почему на macOS сложнее с GPU-LLM в Docker?

Платформа и модель доступа к GPU в контейнерах отличаются от типичного Linux+NVIDIA сервера, поэтому для демонстрации выбран отдельный CPU-путь.

Заключение

Инференс — отдельная инженерная область: от формата модели и контракта API до политик батчинга и управления памятью.

Материалы в `flow/` проходят этот путь последовательно: от обучения и экспорта к серверам и измерениям, затем — к отдельной ветке LLM.